ELEC473 – DIGITAL SYSTEMS DESIGN

Assignment 1 Report

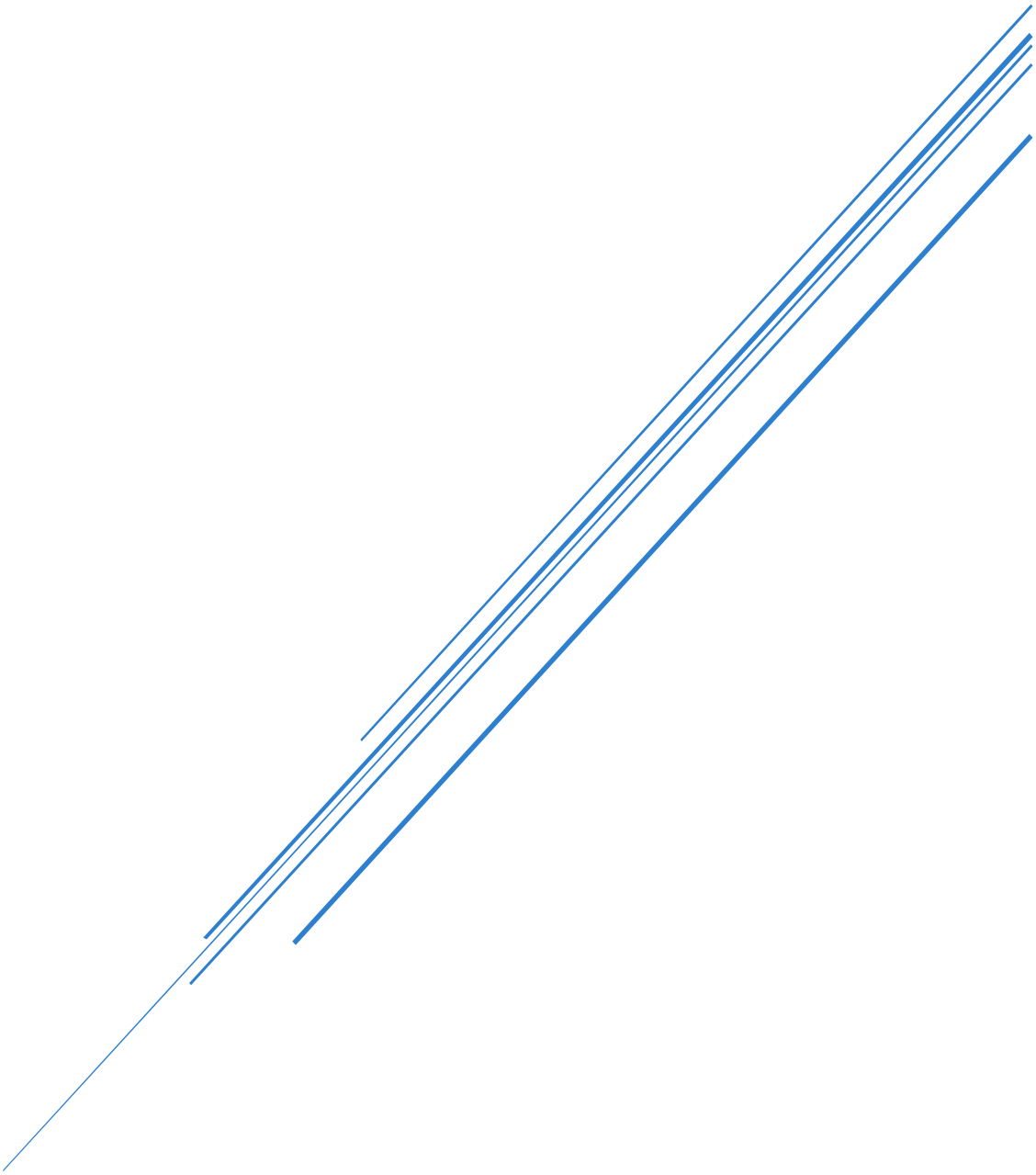

Table of Contents

1. Introduction

1.1. Overview

This report documents the design and implementation of an electronic safe controller of v0.1 developed for ELEC473 Digital Systems Design (Assignments 1 & 2). The aim is to demonstrate the full digital design process — from requirements and top-down conceptual design through ASM charts and Verilog implementation — for a synchronous controller that manages user number entry, locking/unlocking and a programming mode. Assignment 1 concentrates on the conceptual and embodiment design (block diagrams, ASMs, and selected Verilog modules); Assignment 2 extends this work to a complete, tested implementation on the DE2 board using Altera Quartus II.

Design decisions are driven by functional requirements (robust number entry for digits 0–9, reliable lock/unlock behaviour, programming mode, clear LED/HEX display outputs) and non-functional requirements (fully synchronous operation, debounced/deterministic inputs, clear testability and modularity). A top-down methodology is used throughout: requirements → architecture → module decomposition → ASM charts → Verilog implementation → verification. This ensures clarity of specification, justification of design choices, and traceability from requirements to test results.

To make the design manageable and verifiable the system is split into three primary components: the **Mode Controller** (handles restart, mode selection and global state), the **Code Entry Module** (number selection, entry shift and display interface), and the **Code Control Module** (code comparison and unlock behaviour, default code and reprogramming code). The remainder of the report presents block diagrams and ASMs for each module.

1.2. I/O Assignment Table

To maintain consistency with the assignment brief, this report references all inputs and outputs using the assignment key labels provided in the specification. However, the DE2 board uses different physical parameter names for these same signals. For clarity, the table below maps each assignment key to its corresponding DE2 hardware parameter. All architectural diagrams, ASM charts, and documentation will use the assignment keys, while the Verilog implementation will use the DE2 parameters.

This approach ensures that the conceptual design remains aligned with the specification, while the implementation remains accurate to the hardware.

Assignment Keys	Assignment Params
KEYA	KEY3
KEYB	KEY2
KEYC	KEY0
KEYD	KEY1
SWX	SW8
SWM	SW9
RLEDX	LEDR8
GLEDX	LEDG2
X Digits	4
Default X Digits	2019
Display	HEX3-0

Figure 1: I/O Assignment Table

2. System Architecture

2.1. Architecture Overview

The Verilog Lock System is built around three primary functional modules that communicate through a coordinated state-based protocol:

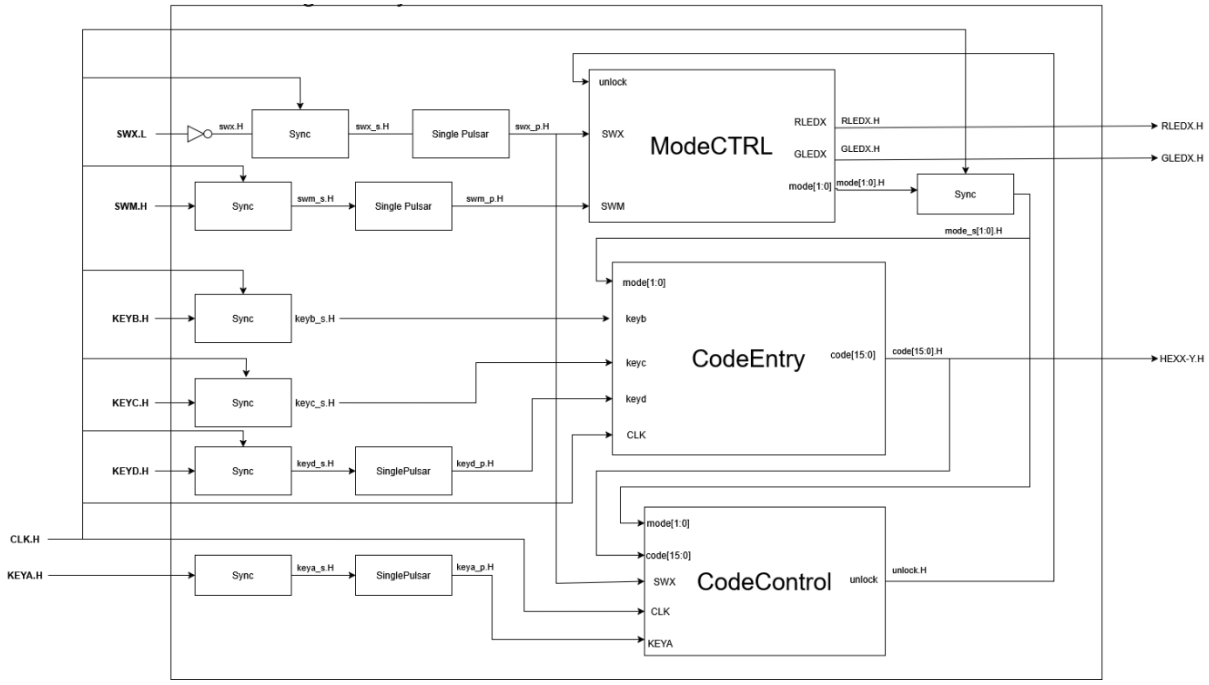

Figure 2: High Level Verilog Lock System Architecture

Module Interaction and Data Flow

- **ModeCTRL** serves as the central orchestrator, maintaining the current operational state and broadcasting mode information to the other modules. It receives control signals (unlock, SWX, SWM) and generates the 2-bit mode signal mode[1:0] that determines system behaviour. This mode signal is synchronized and distributed to both **CodeEntry** and **CodeControl** modules, ensuring all components operate in coordination.
- **CodeEntry** monitors the current mode and accepts user key inputs (KEYB, KEYC, KEYD) to assemble a 16-bit password code. It outputs this assembled code[15:0] to **CodeControl** and also provides hexadecimal display output (HEXX-Y_H) for user feedback. The module's behaviour adapts based on the mode signal—knowing whether it's collecting a password for validation or for programming purposes.
- **CodeControl** receives both the mode information and the assembled code from CodeEntry. Based on the current mode, it performs different operations: in LOCKED mode, it validates the entered code against the stored password and generates an unlock signal back to ModeCTRL if successful; in UNLOCKED_PROGRAMMING mode, it accepts a new password to store. The KEYA input triggers these operations, while SWX provides a reset mechanism to restore the default password.

The critical feedback loop occurs when CodeControl validates a correct password and asserts unlock_H, which feeds back to ModeCTRL. This causes a state transition from LOCKED to UNLOCKED, which then propagates back through the mode signal to inform both CodeEntry and CodeControl of the new system state.

Synchronization and External Interface

The system operates as a fully **synchronous digital design** clocked by CLK_H. All internal state changes and logic operations occur on clock edges, ensuring predictable timing and reliable operation.

External inputs (SWX.L, SWM.H, KEYB.H, KEYC.H, KEYD.H, KEYA.H) are asynchronous—they can change at any time relative to the system clock. To safely integrate these signals into the synchronous design, each input passes through:

1. **Sync (Synchronizer) blocks**: Two-stage flip-flop synchronizers that prevent metastability issues when capturing asynchronous signals
2. **SinglePulsar blocks**: Edge detectors that convert sustained button presses into single-clock-cycle pulses, preventing multiple unintended operations from a single button press

This input conditioning pipeline ensures that by the time signals reach the three main modules, they are properly synchronized to CLK_H and debounced.

External outputs are the LED control signals (RLEDX_H, GLEDX_H) and the hexadecimal display (HEXX-Y_H). These outputs are directly driven by registered signals from ModeCTRL and CodeEntry respectively, ensuring they are glitch-free and synchronized to the system clock.

Coordinated Operation

The beauty of this architecture lies in the mode-based coordination:

- **Mode distribution**: ModeCTRL's mode output creates a shared context for all modules
- **Data path**: CodeEntry → code[15:0] → CodeControl forms the data pipeline
- **Control path**: CodeControl → unlock_H → ModeCTRL → mode[1:0] → (CodeEntry + CodeControl) forms the control feedback loop
- **Synchronous timing**: All state changes occur on clock edges, eliminating race conditions
- **Input safety**: All external asynchronous inputs are synchronized and pulse-conditioned before reaching internal logic

This creates a robust, deterministic system where the three modules operate as a cohesive unit, with clear handshaking protocols mediated by the mode state and synchronized to a common clock domain.

2.2. Mode Controller

The mode controller ASM determines the safe's current access level and updates the LED indicators accordingly. It handles transitions between secure, unlocked, and programming states using a combination of switch and key inputs, ensuring clear visual feedback and consistent system behaviour, to do this Mode Controller manages three high-level operating states of the safe with transitions occurring on rising edge of the CLK input. These States are:

- **LOCKED (00):** Default secure states. RLEDX = 1, GLEDX = 0.
- **UNLOCKED (01):** Safe successfully opened. GLEDX = 1, RLEDX = 0
- **UNLOCKED_PROGRAMMING (10):** Programming mode entered after unlocking. Both LEDs are asserted to indicate this mode.

On system initialization or SWX H we reset to default **LOCKED (00)** State from Locked state transitions can occur as follows:

- **In Locked:**
  - System reset (SWX) places the controller in LOCKED.
  - A valid code entry (internal unlock signal) moves LOCKED → UNLOCKED.
- **In UNLOCKED:**
  - KEYA pressed in UNLOCKED returns the system to LOCKED.
  - SWM pressed in UNLOCKED enters UNLOCKED → UNLOCKED_PROGRAMMING.
- **In UNLOCKED_PROGRAMMING:**
  - KEYA returns the system to LOCKED.
  - SWM returns the system to UNLOCKED.

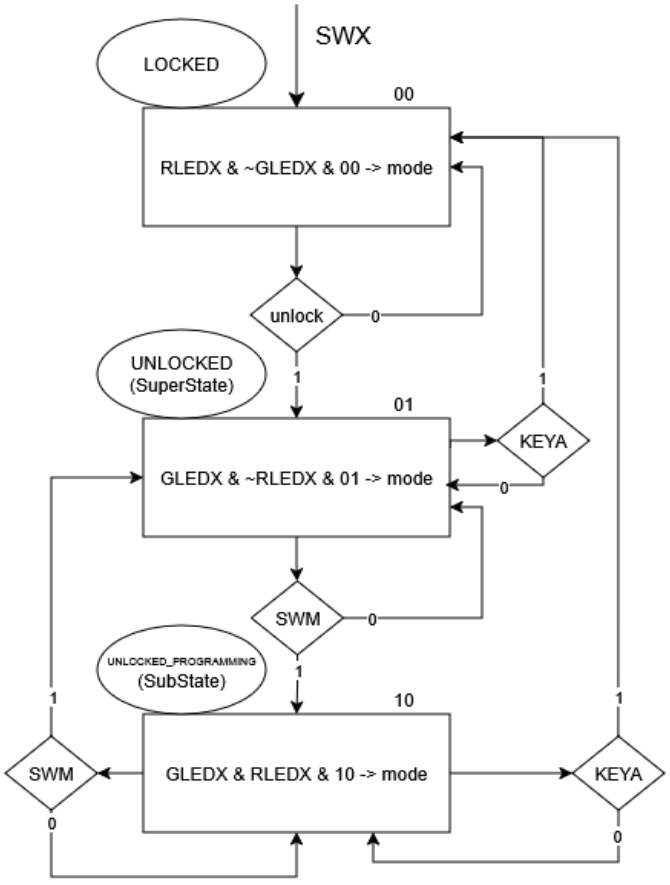

Figure 3 ModeController ASM

2.3. Code Entry Module

Overview

The module shown in Figure 4 implements a code entry interface that allows to input a multi-digit code sequence which is the main function, is also has the ability to reset on mode change.

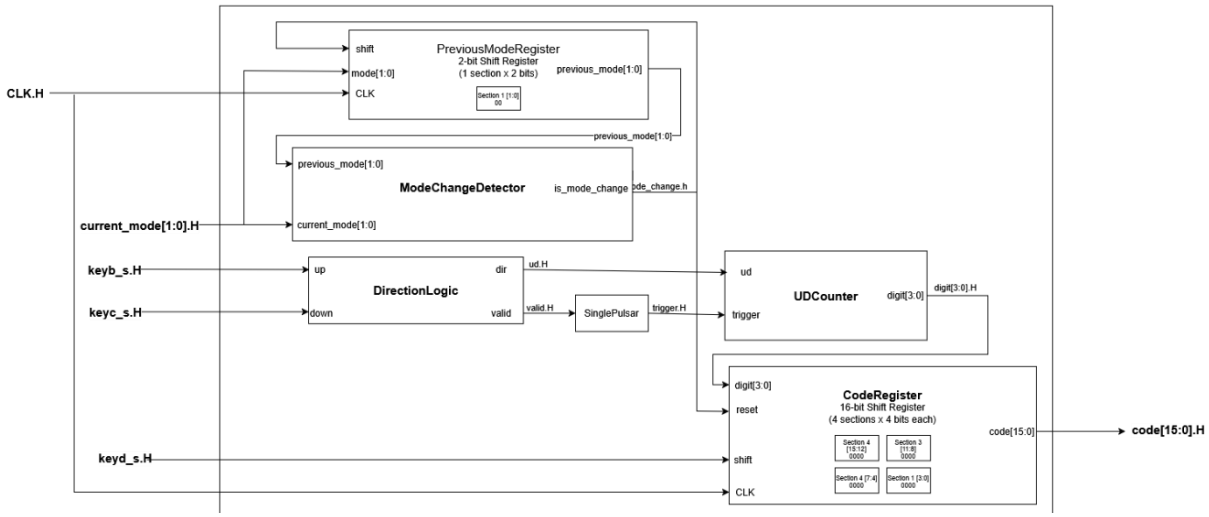

Figure 4: CodeEntry Module Architecture

The CodeEntry Module contains 4 main submodules:

Direction Logic & Counter

The up/down counter is controlled by **KEYB** and **KEYC** buttons a 4-bit synchronous Up/Down counter, where state transitions occur on rising edge of trigger input you see in the UDCounter, which increment or decrement the current digit value being entered. The DirectionLogic block determines the counting direction, while the Single Pulser ensures clean, debounced pulses. The UDCounter outputs the current digit value (digit[3:0]) that the user has selected.

DirectionLogic

Logic Expression

Valid = up ⊕ down

Truth Table

up	down	dir	valid	Explanation
0	0	X	0	No key pressed -> Invalid
0	1	0	1	down pressed
1	0	1	1	up pressed
1	1	X	0	Both pressed -> Invalid

UDCounter

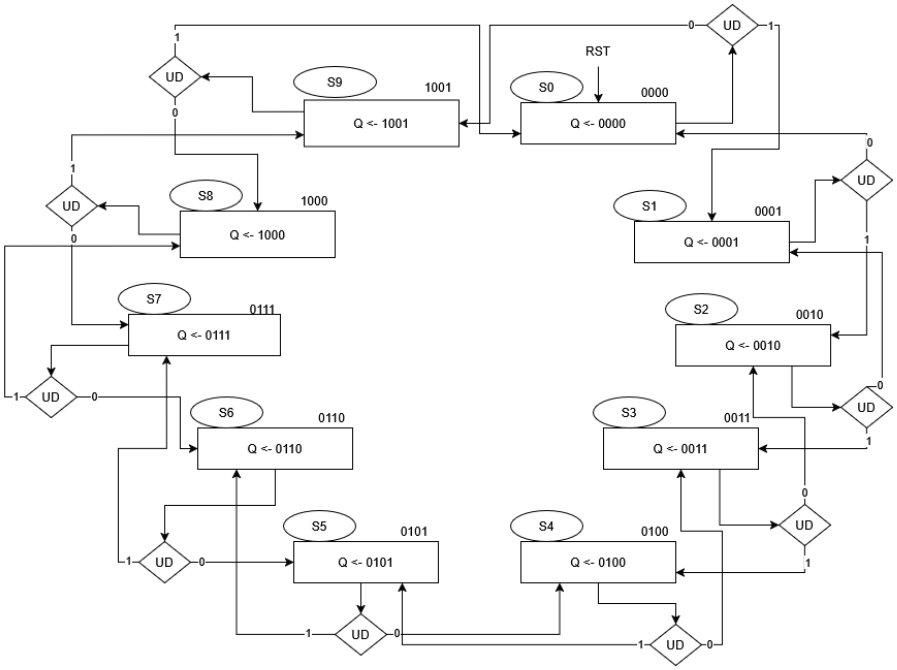

Figure 5: 4-bit updown counter asm for decimal representations 0-9
NOTE: this is my original version, Q is now digit and UD is not lower case ud

Code Registry

A 16-bit shift register organized as 4 sections of 4 bits each, storing the complete 4-digit code. When the user presses keyd_s_H, the currently selected digit is shifted into the register, moving previously entered digits to the left. This allows sequential entry of each code digit, with the final assembled code output as code[15:0].

2.4. Code Control Module

The Code Control Module operates in two modes: Locked (00) and UNLOCKED_PROGRAMMING (10). It manages password validation and programming through five primary inputs:

- **CLK**: Provides clock synchronization for all sequential logic
- **SWX_H**: Reset signal that restores the CurrentCodeRegistry to its default password value of 4'd2019 (binary: 0000 0111 1110 0011)
- **KEYA_H**: Trigger signal for password validation or programming operations
- **code[15:0]**: 16-bit input for password entry or new password programming
- **mode[1:0]**: Mode selection from the main controller

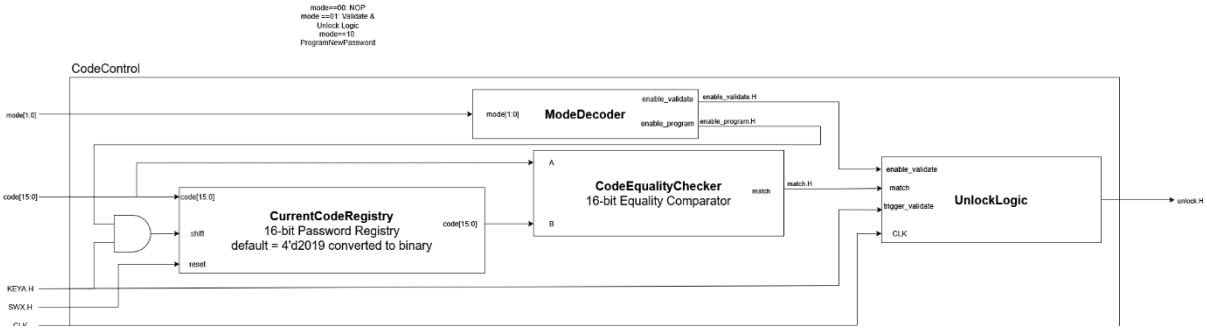

Figure 6: Code Control Module Architecture

Operation Flow:

In **Locked mode (00)**, when KEYA_H is pressed, the ModeDecoder activates the enable_validate signal. This enables the CodeEqualityChecker to compare the input code[15:0] against the value stored in CurrentCodeRegistry. If they match, the checker outputs match=1, which triggers the UnlockLogic block. The resulting unlock_H signal is sent to the mode controller to transition the system to unlocked state.

In **UNLOCKED_PROGRAMMING mode (10)**, when KEYA_H is pressed, the ModeDecoder activates enable_program, allowing a new password from code[15:0] to be written into the CurrentCodeRegistry, updating the stored password for future validation attempts.

3. Discussion

The modular architecture provides clear separation of concerns, with each module handling a distinct functional responsibility. The Mode Controller manages system state, Code Entry handles user input sequencing, and Code Control manages password validation and storage. This modularity makes the design easier to test, debug, and maintain, as each block can be verified independently before system integration.

The fully synchronous design eliminates timing hazards and race conditions. All state changes occur on the rising edge of CLK_H, ensuring predictable behavior. The input synchronization pipeline (Sync + SinglePulsar blocks) safely integrates asynchronous button presses into the synchronous domain, preventing metastability issues that could cause unpredictable behavior.

The use of a centralized mode signal creates a clean coordination mechanism between modules. Rather than complex handshaking protocols, all modules simply observe the current mode and adapt their behavior accordingly. This simplifies the control logic and reduces the potential for timing-dependent bugs.

However, the current design lacks timeout functionality in UNLOCKED and UNLOCKED_PROGRAMMING modes. A real-world safe should automatically return to LOCKED after a period of inactivity for security purposes. This would require adding timer counters to the Mode Controller.

There is no mechanism to prevent brute-force attacks. The system allows unlimited password attempts without delay or lockout. A more secure implementation would include attempt counting and enforced delays after multiple failed attempts.

Error handling is minimal—there's no user feedback for incorrect password attempts beyond simply remaining locked. Adding a temporary LED blink or display message would improve usability.

Verilog implementation & testing was not completed due to overrunning assignment deadline therefore this will need to be handled during assignment 2 of this project.

4. Conclusion

This report has presented the conceptual and embodiment design for an electronic safe controller meeting the ELEC473 Assignment 1 requirements. The top-down design methodology produced a clean three-module architecture with clear interfaces and well-defined responsibilities.

The system successfully addresses the functional requirements: multi-digit code entry via up/down buttons, lock/unlock functionality with LED feedback, programmable combination storage, and a default reset mechanism. The design adheres to best practices for synchronous digital systems with proper input synchronization and single clock domain operation.

Block diagrams and ASM charts provide complete specification of system behaviour, enabling straightforward Verilog implementation. The Code Entry module, required for Assignment 1, has been designed with clear state transitions and is ready for coding and simulation.

Unfortunately for this assignment I ran out of time for implementing modules for this design therefore Assignment 2 will build upon this foundation to deliver a fully functional system both simulated and tested on the DE2 board, with complete Verilog implementation of all modules, comprehensive simulation results, and experimental validation demonstrating reliable operation in all specified modes.